Appointment Scheduling System (Phase 1)

Team: 3 students
Deadline for Phase 1: One week after the midterm exam
Deadline for Phase 2: two week before the final exam

Overall Goal

Students will build an **Appointment Scheduling System** incrementally using **Java 8+**, **Maven**, **JUnit 5**, **JaCoCo**, and **Mockito**, while applying **design patterns** and a **layered architecture**. The project is divided into Agile sprints, where each sprint delivers user stories with clear acceptance criteria.

Setup Requirements (Week 1 – Environment & Tools)

System Stories

US0.1 – Install JDK
As a student, I want to install the latest JDK (>= Java 8) so that I can compile and run Java applications.

Acceptance: - Running `javac -version` on cmd shows version ≥ 1.8

US0.2 – Install IDE
As a student, I want to install an IDE for Java development so that I can develop, debug, and run my code easily.

Acceptance: - IDE launches successfully - A Maven project can be created

US0.3 – Create Maven project
As a student, I want to create a Maven project so that I can manage dependencies, builds, and tests.

Acceptance: - pom.xml exists - mvn test runs successfully.

Functional Stories (Features)

Sprint 1 – Core Scheduling & Authentication [Week 2]

US1.1 – Administrator login
As an administrator, I want to log into the system using my credentials so that I can manage schedules and reservations.

Acceptance: - Valid credentials → login success - Invalid credentials → error message

US1.2 – Administrator logout
As an administrator, I want to log out so that my session is closed securely.

Acceptance: - After logout, admin actions require re-login

US1.3 – View available appointment slots
As a user, I want to view available appointment slots so that I can select a suitable time.

Acceptance: - Only available slots are displayed - Fully booked slots are not selectable

Sprint 2 – Booking & Business Rules [Week 3–4]

US2.1 – Book appointment
As a user, I want to book an appointment for a specific date and time so that I can reserve a visit or meeting.

Acceptance: - Appointment is saved - Status = Confirmed

US2.2 – Enforce visit duration rule
As a system, I want to limit appointment duration so that no booking exceeds the allowed time.

Acceptance: - Maximum duration enforced (e.g., 1/2 hours) - Invalid durations are rejected

US2.3 – Enforce participant limit
As a system, I want to limit the number of participants per appointment so that capacity rules are respected.

Acceptance: - Maximum number of participants enforced - Error shown if limit exceeded

Sprint 3 – Notifications & Mocking [Week 4]

US3.1 – Send appointment reminders
As the system, I want to send reminders for upcoming appointments so that users are notified in advance.

Acceptance: - Reminder message generated - Mock notification service records sent messages in test mode.

Sprint 4 – Advanced Management Rules [Week 5-6]

US4.1 – Modify or cancel appointment
As a user, I want to modify or cancel my upcoming appointment so that I can adjust my schedule.

Acceptance: - Only future appointments can be modified - Slot becomes available again after cancellation

US4.2 – Manage reservations
As an administrator, I want to modify or cancel reservations so that exceptional cases can be handled.

Acceptance: - Only administrators can perform this action

Sprint 5 – Appointment Types & Polymorphism [Week 7–8]

US5.1 – Support multiple appointment types
As a user, I want to choose between different appointment types so that the system supports multiple use cases. [urgent, follow-up, assessment, virtual, in-person, individual, and group appointments]

Acceptance: - Appointment type stored correctly

US5.2 – Apply different rules per type
As a system, I want to apply different rules based on appointment type so that business logic remains flexible.

Acceptance: - Correct rules applied per appointment type

Testing & Mocking Requirements

- Unit testing using **JUnit 5** for core logic (booking rules, validation)
- Code coverage using **JaCoCo**
- Mocking using **Mockito** for:
    - Notification services
    - Time and date handling

Design Patterns

Strategy Pattern

Where: Appointment rule enforcement (duration, capacity, type-specific rules)
Why: Allows adding new rules without modifying core booking logic

```
interface BookingRuleStrategy {
 boolean isValid(Appointment appointment);
}
```

Observer Pattern

Where: Notifications (appointment reminders, updates)
Why: Allows multiple notification channels (email, calendar, SMS)

```
interface Observer {
 void notify(User user, String message);
}
```

Suggested Architecture

Architecture Style: Layered (N-Tier)

1. **Presentation Layer (Optional)**
    - CLI or basic Swing/JavaFX UI
    - Handles user interaction
2. **Application / Service Layer**
    - Coordinates booking, cancellation, notifications
3. **Domain Layer**
    - Entities: Appointment, User, Administrator, Schedule
    - Value Objects: TimeSlot, NotificationMessage
4. **Persistence Layer**
    - In-memory (like Linked list) or database-based storage

Documentation Requirement

All classes, methods, and fields must include **Javadoc comments** using standard syntax (@param, @return, @author, @version).

Finally, UML class diagram.